QUANSER
INNOVATE·EDUCATE

Lab Procedure
Joysticks and Potentiometers

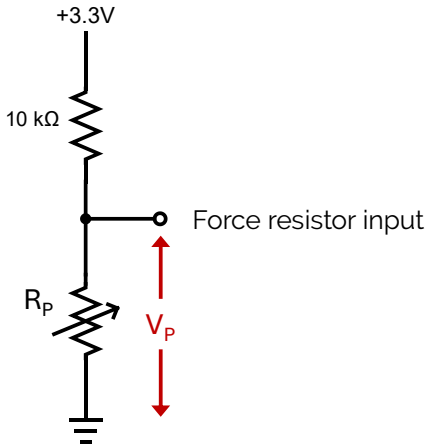
Introduction

Ensure the following:

1. You have reviewed the Application Guide – Joysticks and Potentiometers
2. Make sure the **Mechatronic Sensors Trainer** is out of its case; it is powered using the provided USB cable connected to the PC.
    a. The LED next to the USB C port will stay white after the touch screen starts loading.
    b. The load screen with the Quanser Logo should disappear after a few seconds and you should see some evenly spaced crosses on a black background on the touch screen.
3. Take the potentiometer (100kΩ) and appropriate jumper wires from the Sensors kit.

Exploring Potentiometer Inputs

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**..., and browse to the directory containing the python files and lab procedure for the Joystick and Potentiometers lab (…/sensors_trainer/1_fundamentals/joystick_potentiometer/hardware/python). Open this directory.
2. The following schematic shows the internal circuit in the Sensors Trainer for the force sensing resistor (FSR) input. The circuit is a voltage divider made up of an internal $10k\Omega$ resistor and a second resistor that we connect externally. In this lab, you will connect a **100kΩ** potentiometer with variable resistance R_P, and use the FSR input to measure the voltage V_P across the potentiometer. Write the equation for the voltage V_P across the potentiometer with resistance R_P. Using your equation, what is V_P when $R_P = 0\Omega$? What is V_P when $R_P = 100k\Omega$?



3. Connect the middle and right terminals of the potentiometer using the provided jumper wires to the force resistor input and ground pins, which are the leftmost GPIO pins on the device, marked in the image below:

4. Open joystick.py. Run the script by clicking the play button ▷ at the top right corner of VS code. This will open a **Joystick** scope displaying the readings from the potentiometer and joystick.

5. Rotate the potentiometer knob and observe the **`potentiometer`** signal on the scope, which displays the voltage measured across the potentiometer. How does this signal change when the potentiometer is rotated clockwise or counterclockwise?

6. Stop the script by clicking back on the terminal in Visual Studio Code, then pressing **Ctrl + C**, and then clicking enter when prompted in the terminal.

7. Swap the potentiometer connectors so you are now connecting the left and middle terminals of the potentiometer to the Sensors Trainer.

8. Run the script and repeat step 5. How does this compare to your results from step 5? What is the relationship between the angular position of the potentiometer knob and the voltage? Stop the script.

9. Open the datasheet for the potentiometer, located in the directory …/sensors_trainer/1_fundamentals/joystick_potentiometer. Scroll down to the Tapers section of the datasheet and find the plot corresponding to the B1 potentiometer. What does it indicate about the relationship between the potentiometer shaft angle and the voltage across the potentiometer?

10. Highlight the line after the comment **`# Print potentiometer values`** and press Ctrl + / to uncomment it. Run the script. Rotate the potentiometer knob to its limits in each direction. Read the measured voltage printed to the terminal (ignore the other value **`potAngle`** for now). Do they match the expected voltages at the minimum and maximum resistances given by the equation from step 2? Stop the script.

11. Refer to the datasheet and report the potentiometer's angular range. Given its angular range, how would you use the measured voltage to obtain the angular position of the potentiometer shaft? Write the equation to calculate the angle, θ, given the measured voltage, V_P. In the code, calculate the potentiometer angle and save it in the variable **`potAngle`** after the comment **`# Estimate potentiometer angle`**

12. Run the script. Move the window containing the Joystick scope to the side to uncover the window with the Potentiometer Angle scope. Slowly rotate the potentiometer at a consistent speed between its limits in each direction while observing the estimated angle in the terminal output. Does the estimated angle change consistently with the potentiometer rotation? What happens when the potentiometer is rotated near its limits? How does this compare to when the potentiometer is rotated starting in the middle?

13. Stop the script.

Exploring Joystick Inputs

14. In this section, we will use the joystick located to the right of the LCD screen on the device. In the same file, joystick.py, comment out the line under the comment **`# Print potentiometer values`** by pressing Ctrl + /. Run the script.

15. Move the joystick around to its limits while observing the scope. How do the **`Joystick X`** and **`Joystick Y`** signals change as you move in each direction? What happens if you press down on the joystick? Stop the script.

16. Compare the joystick to the potentiometer. How do they differ in the way that you move them? How does the signal differ? How are they the same?

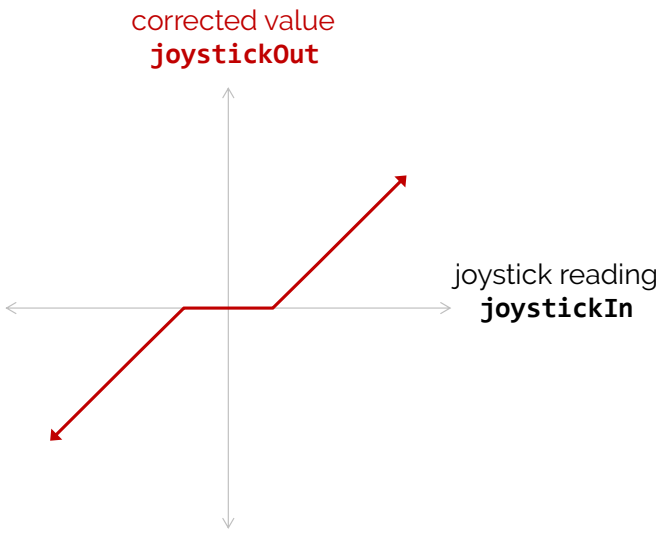
17. The joystick inputs from sensors trainer are saved in the variable **`joystick`**. We can access the readings for each of the X and Y directions by indexing **`joystick[0]`** and **`joystick[1]`**, respectively. These are already saved as **`joystick0`** and **`joystick1`** to

simplify it for you. Under the comment **`# Print joystick readings`**, implement a line of code that prints the X and Y readings to the terminal.

18. Run the script. Move the joystick and then release it to bounce back to the center position. Try this a few times and note the Joystick X and Y signals in the scope whenever the joystick is at the center position. What happens if you slowly return the joystick instead of quickly releasing it? Capture your results in a screenshot. Stop the script.

19. Joysticks are often used to control movement in applications such as video game or aircraft controllers. What should the signals ideally read when the joystick is in the center and why is this important?

20. In your code, implement the **`correct_deadzone`** function to apply a dead zone correction to the joystick reading, **`joystickIn`**. The dead zone bounds are specified by the parameters **`lowerLimit`** and **`upperLimit`**. Outside of the dead zone bounds, the value should increase linearly from 0.



21. In the **`probe.send(name='Joystick',…`** function call, change **`joystick0`** and **`joystick1`** to **`correctedJoy0`** and **`correctedJoy1`** to plot the corrected joystick signals in the scope.

22. Repeat step 18 and take a screenshot of the scope to show that the dead zone correction is working.

23. How would you obtain the angle of the joystick using the Joystick X and Joystick Y readings?

24. Stop, save and close your program.

25. Power OFF the Mechatronic Sensors Trainer by disconnecting its USB C cable, disconnect any external sensors and pack them along the base and the USB cables in the provided case.